

Our Async data type now supports asynchronous computations—we can embed them using the Suspend constructor, which takes an arbitrary Par. This works, but we take this idea one step further and abstract over the choice of type constructor used in Suspend. To do that, we’ll generalize TailRec / Async and parameterize it on some type constructor F rather than use Function0 or Par specifically. We’ll name this more abstract data type Free:

```
sealed trait Free[F[_],A]
case class Return[F[_],A](a: A) extends Free[F,A]
case class Suspend[F[_],A](s: F[A]) extends Free[F,A]
case class FlatMap[F[_],A,B](s: Free[F,A],
                             f: A => Free[F,B]) extends Free[F,B]
```

The difference between Free and TailRec is that Free is parameterized with a type constructor F. TailRec is a special case of Free where F is fixed to be Function0.

The suspension is now of some arbitrary type F rather than Function0.

Then TailRec and Async are simply type aliases:

```
type TailRec[A] = Free[Function0,A]
type Async[A] = Free[Par,A]
```

13.4.1 Reasonably priced monads

The Return and FlatMap constructors witness that this data type is a monad *for any choice of F*, and since they’re exactly the operations required to generate a monad, we say that it’s a *free* monad.⁹



EXERCISE 13.1

Free is a monad for any choice of F. Implement map and flatMap methods on the Free trait, and give the Monad instance for Free[F,_].¹⁰

```
def freeMonad[F[_]]: Monad[({type f[a] = Free[F,a]})#f]
```



EXERCISE 13.2

Implement a specialized tail-recursive interpreter, runTrampoline, for running a Free[Function0,A].

```
@annotation.tailrec
def runTrampoline[A](a: Free[Function0,A]): A
```

⁹ “Free” in this context means *generated freely* in the sense that F itself doesn’t need to have any monadic structure of its own. See the chapter notes for a more formal statement of what “free” means.

¹⁰ Note that we must use the “type lambda” trick discussed in chapter 10 to partially apply the Free type constructor.